

Limitations

What SecureGate does not do yet, or cannot do, and what to do about each. We would rather say it here than have a judge or a user find it.

The merge gate

A pull request can change the workflow that judges it. GitHub runs the pull request's own copy of `.github/workflows/secret-gate.yml`, so a pull request could edit or remove the check. SecureGate itself and its rules come from main, so the rules cannot be loosened that way, but the workflow can. What helps: `.github/CODEOWNERS` names who must review changes to `.github/`, `policy.yml`, `.gitleaks.toml`, `.trufflehog.yml` and `rules/`. To make that review required, tick **Require review from Code Owners** in the ruleset. On a repository with one maintainer that would block every such pull request, because nobody can approve their own, so it stays off until there is a second maintainer. Until then, read every pull request that touches those files with care.

Semgrep and Bandit see only the current code. They read the files a pull request changed, as they are at its end. Gitleaks and TruffleHog read every commit; Semgrep and Bandit do not, so a password typed in one commit and removed in the next is only caught if Gitleaks or TruffleHog know its format.

TruffleHog can only check keys of the providers it knows. For those, it asks the provider whether the key still works (rule 1). ACME Pay, the demos' invented provider, has no address to ask yet, so its tokens always show "Not checked" or "Unverified". A key that cannot be checked is still blocked when its format is dangerous (rule 8).

Semgrep and Bandit are optional. If one of them fails, the scan goes on and the comment says in red that it did not run. Gitleaks and TruffleHog are required: if either fails, the check turns red.

Offline, Semgrep runs only our own rules. Its `p/secrets` rules are downloaded when it runs. Without the internet, it runs `rules/securegate-risky.yml` alone, and the report says so.

Pull requests from forks get no comment. GitHub gives them a read-only token, so SecureGate cannot post its comment or send results to the Security tab. The check still runs and decides.

Fingerprints change on every run on GitHub. A fingerprint is made with a private key; on GitHub there is none, so each run makes a new one, and the same secret gets a different id in each run. A repository secret called `SECUREGATE_HMAC_KEY` would keep ids the same across runs. It is optional.

Semgrep and Bandit are pinned by version, not by checksum. Gitleaks and TruffleHog are checked against their published checksums before they run. Semgrep and Bandit are installed with pip at an exact version, but the packages they depend on can change.

The GitHub Action has the same limits. `action.yml`, the same gate for other repositories (Protect any repository (install.pdf)), shares every limit above, and it judges pull requests only: on any other event it stops with exit code 2.

The runner image is fixed, and will one day be retired. The gate runs on `ubuntu-24.04`, the image it was tested on, rather than `ubuntu-latest`, which moves to a newer Ubuntu over time. GitHub retires old images a few years after their release. When it announces that for 24.04, change `runs-on` in `.github/workflows/secret-gate.yml` to the next image and open one pull request to see the gate pass on it.

Demo pull requests

They need a clean working tree, gh and a sign-in. `securegate demo-pr` refuses to start while `git status` lists changes, so that none of them can end up on GitHub. The demo commits carry your own Git name and email.

Closed demo pull requests stay on GitHub. `make demo-cleanup` closes them and deletes their branches, but GitHub keeps closed pull requests and their commits for good. Every value in them is fake, so that is harmless. Never open one with a real key.

The ruleset can only require the check after it has run once. Open one pull request first (`make demo-clean`), then set the ruleset.

Finding secrets

Known misses. The scorecard in `make test` shows them honestly: with all four scanners, a password inside a `Dockerfile` is still missed, and an AWS secret key next to its key id gets a warning instead of a block. See [Testing \(testing.pdf\)](#).

False alarms on our own code. Scanning SecureGate itself with all four scanners gives 4 warnings, and nothing blocked: Bandit's B105 check takes dictionary keys such as `"acme_token"` in `src/securegate/demo/fakes.py` and `tests/fake_gitleaks.py` for passwords. We leave them as they are rather than rename code to please a scanner; a warning never blocks.

Only what the scanners know. A secret with no known format and no telling name next to it, such as a random string in a variable called `x`, can go unnoticed. That is true of every secret scanner.

The AI agents

The advice can be wrong. It is labelled as advice wherever it appears, and the policy's decision stands.

Code is only sent for reports made on this computer. The value is taken out of the code by its fingerprint, which needs the fingerprint key that made the report. For a report made elsewhere, such as the merge gate's (opened with `make ci-report`), the agents only see the masked findings.

Text in the scanned code can try to steer the agents. A code comment can say "ignore your rules". The agents' tools can only read what SecureGate chose to show, so such text can make the advice misleading, but cannot change a finding, a decision or a file.

They need Azure, and Azure costs money. Each run is small and capped (at most 20 findings, and 8 model calls per agent), but Azure bills for every call. Without Azure, everything else in SecureGate works as before.

Pull requests from forks get no AI advice. GitHub gives them no secrets, so the merge gate cannot reach Azure for them; their comment says the agents were not asked.